SIMATS ENGINEERING

ASSESSMENT TOOL 2: Schema Analysis and Viva Voce

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0501

COURSE OUTCOME COVERED

CO2: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Assessment Tool Weightage: 35%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 20**
Question Count: 2

Student Name: Answar Vijes D Register No: 192324028 Code of Conduct

I Answar Vijes D / 192324028 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Schema Analysis and Viva voce Rubric

Identification of Entities and Attributes (4 Marks)	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Answer Sheet - Schema Analysis and Viva Voce

Course: Query Processing for Data Science (DSA0501) | Student: Manikanda Sabari S | Reg No: 192324312

Notes: All programs are written in Python 3 with the sqlite3 module. The tables follow the schema given in the paper; sample rows are inserted so each query produces output. Outputs were produced by running the code.

Question 1: Student Course Registration Database (10 Marks)

Task: A university maintains the following database schema: Student (Student_ID, Student_Name, Department, Year) Course (Course_ID, Course_Name, Credits) Enrollment (Enrollment_ID, Student_ID, Course_ID, Semester) Here, Student_ID in the Enrollment table is a foreign key referencing the Student table, and Course_ID is a foreign key referencing the Course table. Analyze the above schema and write a Python program using SQLite/MySQL/PostgreSQL to connect to the database and retrieve the names of all students along with the courses they are enrolled in during the "Odd 2026" semester using an appropriate SQL JOIN query.
Approach: Join the three tables through the junction table Enrollment and filter on Semester = 'Odd 2026' with a parameterised WHERE clause.

Schema analysis:

```
Schema analysis
 Student(Student_ID PK, Student_Name, Department, Year)
 Course(Course_ID PK, Course_Name, Credits)
 Enrollment(Enrollment_ID PK, Student_ID FK, Course_ID FK, Semester)

 Relationship: Student M --- N Course. It is resolved by the
 Enrollment (junction) table, which holds both foreign keys.
 Join path: Student.Student_ID = Enrollment.Student_ID
            Enrollment.Course_ID = Course.Course_ID
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Student(Student_ID INTEGER PRIMARY KEY, Student_Name TEXT NOT NULL, Department TEXT, Year
    INTEGER);
CREATE TABLE Course(Course_ID INTEGER PRIMARY KEY, Course_Name TEXT NOT NULL, Credits INTEGER);
CREATE TABLE Enrollment(Enrollment_ID INTEGER PRIMARY KEY,
    Student_ID INTEGER REFERENCES Student(Student_ID),
    Course_ID  INTEGER REFERENCES Course(Course_ID), Semester TEXT);
INSERT INTO Student VALUES
    (1,'Aarthi','CSE',2),(2,'Bharath','ECE',3),(3,'Chandru','IT',1),(4,'Deepa','CSE',4);
INSERT INTO Course VALUES (101,'Database Systems',4),(102,'Python Programming',3),(103,'Data
    Structures',4),(104,'Digital Electronics',3);
INSERT INTO Enrollment VALUES (1,1,101,'Odd 2026'),(2,1,102,'Odd 2026'),(3,2,104,'Odd 2026'),
                              (4,3,103,'Even 2025'),(5,4,101,'Odd 2026'),(6,4,103,'Even 2025');
""")

query = """
SELECT s.Student_Name, c.Course_Name
FROM Student s
JOIN Enrollment e ON s.Student_ID = e.Student_ID
JOIN Course c     ON e.Course_ID = c.Course_ID
WHERE e.Semester = ?
ORDER BY s.Student_Name"""

cur.execute(query, ("Odd 2026",))
show(cur)
con.close()
```

Output:

```
Student_Name | Course_Name
------------------------------------------------------------
Aarthi | Database Systems
Aarthi | Python Programming
Bharath | Digital Electronics
Deepa | Database Systems
```

Viva voce - likely questions:

Q: Why is Enrollment needed?
A: Student and Course have a many-to-many relationship; a junction table with two foreign keys converts it into two one-to-many relationships.

Q: Which join is used?
A: INNER JOIN, because only students with a matching enrollment record should appear.

Q: Why use ? placeholders?
A: Parameterised queries prevent SQL injection and let SQLite reuse the compiled statement.

Question 2: Hospital Management Database (10 Marks)

Task: A hospital maintains the following database schema: Doctor (Doctor_ID, Doctor_Name, Specialization) Patient (Patient_ID, Patient_Name, Age) Appointment (Appointment_ID, Patient_ID, Doctor_ID, Appointment_Date) Analyze the schema and write a Python program to connect to the database and display the patient names, doctor names, and appointment dates for all appointments scheduled after 1 January 2026.
Approach: Join Appointment with Patient and Doctor and filter Appointment_Date > '2026-01-01'. Dates are stored as ISO text (YYYY-MM-DD), so string comparison is also a correct date comparison.

Schema analysis:

```
Schema analysis
 Doctor(Doctor_ID PK, Doctor_Name, Specialization)
 Patient(Patient_ID PK, Patient_Name, Age)
 Appointment(Appointment_ID PK, Patient_ID FK, Doctor_ID FK, Appointment_Date)

 Relationship: Doctor M --- N Patient, resolved by Appointment.
 Join path: Appointment.Patient_ID = Patient.Patient_ID
            Appointment.Doctor_ID  = Doctor.Doctor_ID
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Doctor(Doctor_ID INTEGER PRIMARY KEY, Doctor_Name TEXT NOT NULL, Specialization TEXT);
CREATE TABLE Patient(Patient_ID INTEGER PRIMARY KEY, Patient_Name TEXT NOT NULL, Age INTEGER);
CREATE TABLE Appointment(Appointment_ID INTEGER PRIMARY KEY,
    Patient_ID INTEGER REFERENCES Patient(Patient_ID),
    Doctor_ID  INTEGER REFERENCES Doctor(Doctor_ID), Appointment_Date TEXT);
INSERT INTO Doctor VALUES (1,'Dr. Anand','Cardiology'),(2,'Dr. Meena','Pediatrics'),(3,'Dr.
    Rao','Orthopedics');
INSERT INTO Patient VALUES (1,'Kamala',67),(2,'Rajesh',45),(3,'Baby Ria',4),(4,'Sundar',72);
INSERT INTO Appointment VALUES
    (1,1,1,'2025-12-20'),(2,2,3,'2026-01-15'),(3,3,2,'2026-03-02'),(4,4,1,'2026-01-01');
""")

query = """
SELECT p.Patient_Name, d.Doctor_Name, a.Appointment_Date
FROM Appointment a
JOIN Patient p ON a.Patient_ID = p.Patient_ID
JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID
WHERE a.Appointment_Date > ?
ORDER BY a.Appointment_Date"""

cur.execute(query, ("2026-01-01",))
show(cur)
con.close()
```

Output:

```
Patient_Name | Doctor_Name | Appointment_Date
------------------------------------------------------------
Rajesh | Dr. Rao | 2026-01-15
Baby Ria | Dr. Meena | 2026-03-02
```

Viva voce - likely questions:

Q: Why is the appointment on 2026-01-01 not shown?
A: The condition is 'after 1 January 2026' (strictly greater), so that date itself is excluded.

Q: Which keys does Appointment contain?
A: Its own primary key plus foreign keys Patient_ID and Doctor_ID.

Q: How does SQLite store dates?
A: As TEXT, REAL or INTEGER; the ISO-8601 TEXT format sorts correctly.

Question 3: Online Shopping Database (10 Marks)

Task: An e-commerce company uses the following schema: Customer (Customer_ID, Customer_Name, City) Product (Product_ID, Product_Name, Price) Orders (Order_ID, Customer_ID, Product_ID, Quantity, Order_Date) Analyze the schema and write a Python program to retrieve the customer name, product name, quantity
Approach: Join Orders with Customer and Product; the order value is calculated in SQL as Price * Quantity.

Schema analysis:

```
Schema analysis
 Customer(Customer_ID PK, Customer_Name, City)
 Product(Product_ID PK, Product_Name, Price)
 Orders(Order_ID PK, Customer_ID FK, Product_ID FK, Quantity, Order_Date)

 Relationship: Customer M --- N Product, resolved by Orders.
 Note: the question text is cut off after "quantity" in the paper, so the
 query returns customer name, product name, quantity and order date (plus the
 order value = Price x Quantity) for every order.
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Customer(Customer_ID INTEGER PRIMARY KEY, Customer_Name TEXT NOT NULL, City TEXT);
CREATE TABLE Product(Product_ID INTEGER PRIMARY KEY, Product_Name TEXT NOT NULL, Price REAL);
CREATE TABLE Orders(Order_ID INTEGER PRIMARY KEY,
    Customer_ID INTEGER REFERENCES Customer(Customer_ID),
    Product_ID INTEGER REFERENCES Product(Product_ID), Quantity INTEGER, Order_Date TEXT);
INSERT INTO Customer VALUES (1,'Arun','Chennai'),(2,'Bhavana','Madurai'),(3,'Charan','Salem');
INSERT INTO Product VALUES (1,'Laptop',55000),(2,'Mouse',450),(3,'Keyboard',900);
INSERT INTO Orders VALUES
    (1,1,1,1,'2026-02-10'),(2,2,2,3,'2026-02-12'),(3,3,3,2,'2026-03-01'),(4,1,2,2,'2026-03-05');
""")

query = """
SELECT c.Customer_Name, p.Product_Name, o.Quantity, o.Order_Date, p.Price * o.Quantity AS Order_Value
FROM Orders o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Product p ON o.Product_ID = p.Product_ID
ORDER BY o.Order_ID"""

cur.execute(query)
s h o w ( c u r )
con.close()
```

Output:

```
Customer_Name | Product_Name | Quantity | Order_Date | Order_Value
------------------------------------------------------------
Arun | Laptop | 1 | 2026-02-10 | 55000.0
Bhavana | Mouse | 3 | 2026-02-12 | 1350.0
Charan | Keyboard | 2 | 2026-03-01 | 1800.0
Arun | Mouse | 2 | 2026-03-05 | 900.0
```

Viva voce - likely questions:

Q: What does the Orders table represent?
A: It is the junction (associative) table between Customer and Product and stores the order facts such as Quantity and Order_Date.

Q: Where should Price be stored?
A: In Product, to avoid redundancy; Order_Value is derived with Price * Quantity.

Q: What normal form is this schema in?
A: Third normal form: every non-key attribute depends only on its table's key.

Question 4: Hospital Management Database (10 Marks)

Task: A hospital maintains the following database schema: Doctor (Doctor_ID, Doctor_Name, Specialization) Patient (Patient_ID, Patient_Name, Age) Appointment (Appointment_ID, Patient_ID, Doctor_ID, Appointment_Date) Analyze the schema and write a Python program to retrieve the doctor name, patient name, and appointment date for all appointments scheduled during the current month.
Approach: SQLite's strftime('%Y-%m', date) is compared with strftime('%Y-%m','now') so that only appointments in the current month are returned. The sample dates are generated relative to today's date so the query always has data to show.

Schema analysis:

```
Schema analysis
 Doctor(Doctor_ID PK, Doctor_Name, Specialization)
 Patient(Patient_ID PK, Patient_Name, Age)
 Appointment(Appointment_ID PK, Patient_ID FK, Doctor_ID FK, Appointment_Date)

 Relationship: Doctor M --- N Patient, resolved by Appointment.
 Join path: Appointment.Patient_ID = Patient.Patient_ID
            Appointment.Doctor_ID  = Doctor.Doctor_ID
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

from datetime import date, timedelta
today = date.today()
this_month = [today.replace(day=d).isoformat() for d in (1, 10, 25)]
last_month = (today.replace(day=1) - timedelta(days=5)).isoformat()       # an earlier month

cur.executescript("""
CREATE TABLE Doctor(Doctor_ID INTEGER PRIMARY KEY, Doctor_Name TEXT NOT NULL, Specialization TEXT);
CREATE TABLE Patient(Patient_ID INTEGER PRIMARY KEY, Patient_Name TEXT NOT NULL, Age INTEGER);
CREATE TABLE Appointment(Appointment_ID INTEGER PRIMARY KEY,
    Patient_ID INTEGER REFERENCES Patient(Patient_ID),
    Doctor_ID  INTEGER REFERENCES Doctor(Doctor_ID), Appointment_Date TEXT);
INSERT INTO Doctor VALUES (1,'Dr. Anand','Cardiology'),(2,'Dr. Meena','Pediatrics'),(3,'Dr.
    Rao','Orthopedics');
INSERT INTO Patient VALUES (1,'Kamala',67),(2,'Rajesh',45),(3,'Baby Ria',4),(4,'Sundar',72);
""")
cur.executemany("INSERT INTO Appointment VALUES (?,?,?,?)",
    [(1, 1, 1, this_month[0]), (2, 2, 3, this_month[1]), (3, 3, 2, this_month[2]), (4, 4, 1,
        last_month)])

query = """
SELECT d.Doctor_Name, p.Patient_Name, a.Appointment_Date
FROM Appointment a
JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID
JOIN Patient p ON a.Patient_ID = p.Patient_ID
WHERE strftime('%Y-%m', a.Appointment_Date) = strftime('%Y-%m', 'now')
ORDER BY a.Appointment_Date"""

print("Current month:", today.strftime("%B %Y"), "\n")
cur.execute(query)
show(cur)
con.close()
```

Output:

```
Current month: November 2026

Doctor_Name | Patient_Name | Appointment_Date
------------------------------------------------------------
Dr. Anand | Kamala | 2026-11-01
Dr. Rao | Rajesh | 2026-11-10
Dr. Meena | Baby Ria | 2026-11-25
```

Viva voce - likely questions:

Q: How is the 'current month' found?
A: strftime('%Y-%m','now') returns the year-month of the system date, and it is compared with the same format of the stored date.

Q: Why generate sample dates in Python?
A: So the demo always contains appointments in the real current month.

Q: Which function would MySQL use?
A: MONTH(Appointment_Date) = MONTH(CURDATE()) AND YEAR(...) = YEAR(CURDATE()).

Question 5: University Course Registration Database (10 Marks)

Task: A university maintains the following database schema: Student (Student_ID, Student_Name, Department) Course (Course_ID, Course_Name, Credits) Enrollment (Enrollment_ID, Student_ID, Course_ID, Semester) Analyze the schema and write a Python program to retrieve the student name, course name, and semester for all students enrolled in courses carrying more than 3 credits.
Approach: Three-table INNER JOIN with the filter Credits > 3 on the Course table.

Schema analysis:

```
Schema analysis
 Student(Student_ID PK, Student_Name, Department)
 Course(Course_ID PK, Course_Name, Credits)
 Enrollment(Enrollment_ID PK, Student_ID FK, Course_ID FK, Semester)

 Relationship: Student M --- N Course through Enrollment.
 Filter: Course.Credits > 3 (the filter column lives in Course).
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Student(Student_ID INTEGER PRIMARY KEY, Student_Name TEXT NOT NULL, Department TEXT);
CREATE TABLE Course(Course_ID INTEGER PRIMARY KEY, Course_Name TEXT NOT NULL, Credits INTEGER);
CREATE TABLE Enrollment(Enrollment_ID INTEGER PRIMARY KEY,
    Student_ID INTEGER REFERENCES Student(Student_ID),
    Course_ID  INTEGER REFERENCES Course(Course_ID), Semester TEXT);
INSERT INTO Student VALUES (1,'Aarthi','CSE'),(2,'Bharath','ECE'),(3,'Chandru','IT');
INSERT INTO Course VALUES (101,'Database Systems',4),(102,'Python Programming',3),(103,'Machine
    Learning',5),(104,'Digital Electronics',3);
INSERT INTO Enrollment VALUES (1,1,101,'Odd 2026'),(2,1,102,'Odd 2026'),(3,2,104,'Odd 2026'),
                              (4,3,103,'Even 2026'),(5,2,101,'Even 2026');
""")

query = """
SELECT s.Student_Name, c.Course_Name, e.Semester, c.Credits
FROM Student s
JOIN Enrollment e ON s.Student_ID = e.Student_ID
JOIN Course c    ON e.Course_ID = c.Course_ID
WHERE c.Credits > 3
ORDER BY s.Student_Name"""

cur.execute(query)
s h o w ( c u r )
con.close()
```

Output:

```
Student_Name | Course_Name | Semester | Credits
------------------------------------------------------------
Aarthi | Database Systems | Odd 2026 | 4
Bharath | Database Systems | Even 2026 | 4
Chandru | Machine Learning | Even 2026 | 5
```

Viva voce - likely questions:

Q: Why are 3-credit courses excluded?
A: The condition is Credits > 3 (strictly more than 3).

Q: Could the same result be written without JOIN?
A: Yes, with nested subqueries or comma joins, but explicit JOIN ... ON is clearer and is the standard.

Q: What if a student has no enrollments?
A: An INNER JOIN omits them; a LEFT JOIN from Student would keep them with NULL course values.

Question 6: Banking Database (10 Marks)

Task: A bank maintains the following database schema: Customer (Customer_ID, Customer_Name, City) Account (Account_No, Customer_ID, Account_Type, Balance) Transaction (Transaction_ID, Account_No, Transaction_Date, Amount) Analyze the schema and write a Python program to retrieve the customer name, account number, and balance for customers whose account balance exceeds Rs.1,00,000.
Approach: Join Customer and Account and filter Balance > 100000 (Rs. 1,00,000 in Indian number format).

Schema analysis:

```
Schema analysis
 Customer(Customer_ID PK, Customer_Name, City)
 Account(Account_No PK, Customer_ID FK, Account_Type, Balance)
 Transaction(Transaction_ID PK, Account_No FK, Transaction_Date, Amount)

 Relationships: Customer 1 --- N Account; Account 1 --- N Transaction.
 This query needs only Customer and Account.
 Note: Transaction is a reserved word in SQL, so it must be quoted
 ("Transaction") when the table is created.
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Customer(Customer_ID INTEGER PRIMARY KEY, Customer_Name TEXT NOT NULL, City TEXT);
CREATE TABLE Account(Account_No INTEGER PRIMARY KEY,
    Customer_ID INTEGER REFERENCES Customer(Customer_ID), Account_Type TEXT, Balance REAL);
CREATE TABLE "Transaction"(Transaction_ID INTEGER PRIMARY KEY,
    Account_No INTEGER REFERENCES Account(Account_No), Transaction_Date TEXT, Amount REAL);
INSERT INTO Customer VALUES
    (1,'Arun','Chennai'),(2,'Bhavana','Madurai'),(3,'Charan','Salem'),(4,'Divya','Trichy');
INSERT INTO Account VALUES (1001,1,'Savings',250000),(1002,2,'Current',85000),
                           (1003,3,'Savings',100000),(1004,4,'Savings',1250000.50);
INSERT INTO "Transaction" VALUES (1,1001,'2026-01-05',5000),(2,1004,'2026-01-07',20000);
""")

query = """
SELECT c.Customer_Name, a.Account_No, a.Balance
FROM Customer c
JOIN Account a ON c.Customer_ID = a.Customer_ID
WHERE a.Balance > 100000
ORDER BY a.Balance DESC"""

cur.execute(query)
show(cur)
con.close()
```

Output:

```
Customer_Name | Account_No | Balance
------------------------------------------------------------
Divya | 1004 | 1250000.5
Arun | 1001 | 250000.0
```

Viva voce - likely questions:

Q: Why is Charan (exactly 1,00,000) not listed?
A: The condition is 'exceeds', i.e. Balance > 100000.

Q: Why quote the Transaction table?
A: TRANSACTION is a reserved keyword in SQL.

Q: What is the cardinality between Customer and Account?
A: One-to-many: a customer may hold several accounts; each account belongs to one customer.

Question 7: Library Management Database (10 Marks)

Task: A library maintains the following database schema: Member (Member_ID, Member_Name, Phone) Book (Book_ID, Book_Title, Author) Issue (Issue_ID, Member_ID, Book_ID, Issue_Date) Analyze the schema and write a Python program to retrieve the member name, book title, and issue date for all books issued after 1 January 2026.
Approach: Join Issue with Member and Book and keep rows where Issue_Date > '2026-01-01'.

Schema analysis:

```
Schema analysis
 Member(Member_ID PK, Member_Name, Phone)
 Book(Book_ID PK, Book_Title, Author)
 Issue(Issue_ID PK, Member_ID FK, Book_ID FK, Issue_Date)

 Relationship: Member M --- N Book, resolved by Issue.
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Member(Member_ID INTEGER PRIMARY KEY, Member_Name TEXT NOT NULL, Phone TEXT);
CREATE TABLE Book(Book_ID INTEGER PRIMARY KEY, Book_Title TEXT NOT NULL, Author TEXT);
CREATE TABLE Issue(Issue_ID INTEGER PRIMARY KEY,
    Member_ID INTEGER REFERENCES Member(Member_ID),
    Book_ID   INTEGER REFERENCES Book(Book_ID), Issue_Date TEXT);
I N S E R T       I N T O       M e m b e r       V A L U E S
    (1,'Aarthi','9876500001'),(2,'Bharath','9876500002'),(3,'Chandru','9876500003');
INSERT INTO Book VALUES (1,'Python Basics','R. Kumar'),(2,'Database Systems','Elmasri'),(3,'Data
    Wrangling','J. Kazil');
INSERT INTO Issue VALUES
    (1,1,1,'2025-12-28'),(2,2,2,'2026-01-10'),(3,3,3,'2026-02-14'),(4,1,2,'2026-03-03');
""")

query = """
SELECT m.Member_Name, b.Book_Title, i.Issue_Date
FROM Issue i
JOIN Member m ON i.Member_ID = m.Member_ID
JOIN Book  b ON i.Book_ID  = b.Book_ID
WHERE i.Issue_Date > ?
ORDER BY i.Issue_Date"""

cur.execute(query, ("2026-01-01",))
show(cur)
con.close()
```

Output:

```
Member_Name | Book_Title | Issue_Date
------------------------------------------------------------
Bharath | Database Systems | 2026-01-10
Chandru | Data Wrangling | 2026-02-14
Aarthi | Database Systems | 2026-03-03
```

Viva voce - likely questions:

Q: What role does the Issue table play?
A: It is the transaction/junction table linking members and books and storing Issue_Date.

Q: How would you find books never issued?
A: LEFT JOIN Book to Issue and filter Issue.Issue_ID IS NULL.

Q: Which constraint stops issuing to a non-existing member?
A: The foreign key Member_ID REFERENCES Member(Member_ID) with PRAGMA foreign_keys = ON.

Question 8: Hotel Reservation Database (10 Marks)

Task: A hotel maintains the following database schema: Customer (Customer_ID, Customer_Name, City) Room (Room_ID, Room_Type, Rent) Reservation (Reservation_ID, Customer_ID, Room_ID, Check_In_Date, Check_Out_Date) Analyze the schema and write a Python program to retrieve the customer name, room type, and check-in date for customers who have reserved Deluxe rooms.
Approach: Join Reservation with Customer and Room and filter Room_Type = 'Deluxe'.

Schema analysis:

```
Schema analysis
 Customer(Customer_ID PK, Customer_Name, City)
 Room(Room_ID PK, Room_Type, Rent)
 Reservation(Reservation_ID PK, Customer_ID FK, Room_ID FK,
             Check_In_Date, Check_Out_Date)

 Relationship: Customer M --- N Room, resolved by Reservation.
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Customer(Customer_ID INTEGER PRIMARY KEY, Customer_Name TEXT NOT NULL, City TEXT);
CREATE TABLE Room(Room_ID INTEGER PRIMARY KEY, Room_Type TEXT, Rent REAL);
CREATE TABLE Reservation(Reservation_ID INTEGER PRIMARY KEY,
    Customer_ID INTEGER REFERENCES Customer(Customer_ID),
    Room_ID INTEGER REFERENCES Room(Room_ID), Check_In_Date TEXT, Check_Out_Date TEXT);
INSERT INTO Customer VALUES (1,'Aarthi','Chennai'),(2,'Bharath','Madurai'),(3,'Chandru','Salem');
INSERT INTO Room VALUES
    (201,'Deluxe',4500),(202,'Standard',2500),(203,'Suite',8000),(204,'Deluxe',4800);
INSERT INTO Reservation VALUES (1,1,201,'2026-04-01','2026-04-03'),(2,2,202,'2026-04-05','2026-04-06'),
                               (3,3,204,'2026-04-10','2026-04-12'),(4,2,203,'2026-04-15','2026-04-18');
""")

query = """
SELECT c.Customer_Name, r.Room_Type, v.Check_In_Date
FROM Reservation v
JOIN Customer c ON v.Customer_ID = c.Customer_ID
JOIN Room r     ON v.Room_ID     =  r.Room_ID
WHERE r.Room_Type = ?
ORDER BY v.Check_In_Date"""

cur.execute(query, ("Deluxe",))
show(cur)
con.close()
```

Output:

```
Customer_Name | Room_Type | Check_In_Date
------------------------------------------------------------
Aarthi | Deluxe | 2026-04-01
Chandru | Deluxe | 2026-04-10
```

Viva voce - likely questions:

Q: Is Room_Type a good candidate for a separate table?
A: Yes, in a stricter design a Room_Type table would remove repeated text values.

Q: How is the stay length calculated?
A: julianday(Check_Out_Date) - julianday(Check_In_Date).

Q: Why is the filter value passed as a parameter?
A: It avoids SQL injection and makes the query reusable for other room types.

Question 9: Employee Project Management Database (10 Marks)

Task: A software company maintains the following database schema: Employee (Employee_ID, Employee_Name, Department) Project (Project_ID, Project_Name, Budget) Works_On (Employee_ID, Project_ID, Role) Analyze the schema and write a Python program to retrieve the employee name, project name, and role of employees working on projects with a budget greater than Rs.50,00,000.
Approach: Join Works_On with Employee and Project and filter Budget > 5000000 (Rs. 50,00,000).

Schema analysis:

```
Schema analysis
 Employee(Employee_ID PK, Employee_Name, Department)
 Project(Project_ID PK, Project_Name, Budget)
 Works_On(Employee_ID FK, Project_ID FK, Role)
   - composite primary key (Employee_ID, Project_ID)

 Relationship: Employee M --- N Project, resolved by Works_On.
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Employee(Employee_ID INTEGER PRIMARY KEY, Employee_Name TEXT NOT NULL, Department TEXT);
CREATE TABLE Project(Project_ID INTEGER PRIMARY KEY, Project_Name TEXT NOT NULL, Budget REAL);
CREATE TABLE Works_On(
    Employee_ID INTEGER REFERENCES Employee(Employee_ID),
    Project_ID INTEGER REFERENCES Project(Project_ID), Role TEXT,
    PRIMARY KEY (Employee_ID, Project_ID));
INSERT INTO Employee VALUES (1,'Arun','IT'),(2,'Bhavana','IT'),(3,'Charan','Finance'),(4,'Divya','IT');
INSERT INTO Project VALUES (1,'Banking App',7500000),(2,'Website Redesign',3000000),(3,'Data
    Platform',12000000);
INSERT INTO Works_On VALUES
    (1,1,'Lead'),(2,1,'Developer'),(3,2,'Analyst'),(4,3,'Architect'),(1,3,'Developer');
""")

query = """
SELECT e.Employee_Name, p.Project_Name, w.Role
FROM Works_On w
JOIN Employee e ON w.Employee_ID = e.Employee_ID
JOIN Project p ON w.Project_ID = p.Project_ID
WHERE p.Budget > 5000000
ORDER BY p.Project_Name, e.Employee_Name"""

cur.execute(query)
s h o w ( c u r )
con.close()
```

Output:

```
Employee_Name | Project_Name | Role
------------------------------------------------------------
Arun | Banking App | Lead
Bhavana | Banking App | Developer
Arun | Data Platform | Developer
Divya | Data Platform | Architect
```

Viva voce - likely questions:

Q: Why does Works_On have a composite key?
A: An employee can work on many projects and a project has many employees; the pair (Employee_ID, Project_ID) is unique.

Q: Where is Role stored and why?
A: In Works_On, because it depends on both the employee and the project (an attribute of the relationship).

Q: How is Rs. 50,00,000 written in code?
A: As 5000000 - no commas are allowed in numeric literals.

Question 10: Airline Reservation Database (10 Marks)

Task: An airline company maintains the following database schema: Passenger (Passenger_ID, Passenger_Name, City) Flight (Flight_ID, Flight_Name, Destination) Booking (Booking_ID, Passenger_ID, Flight_ID, Travel_Date) Analyze the schema and write a Python program to retrieve the passenger name, flight name, destination, and travel date for all passengers travelling to Delhi.
Approach: Join Booking with Passenger and Flight and filter Destination = 'Delhi'.

Schema analysis:

```
Schema analysis
 Passenger(Passenger_ID PK, Passenger_Name, City)
 Flight(Flight_ID PK, Flight_Name, Destination)
 Booking(Booking_ID PK, Passenger_ID FK, Flight_ID FK, Travel_Date)

 Relationship: Passenger M --- N Flight, resolved by Booking.
```

Program:

```
import sqlite3

con = sqlite3.connect(":memory:") # use sqlite3.connect('file.db') for a file
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON")

def show(cur):                          # print query result with column headings
    cols = [c[0] for c in cur.description]
    print(" | ".join(cols))
    print("-" * 60)
    for row in cur.fetchall():
        print(" | ".join(str(v) for v in row))

cur.executescript("""
CREATE TABLE Passenger(Passenger_ID INTEGER PRIMARY KEY, Passenger_Name TEXT NOT NULL, City TEXT);
CREATE TABLE Flight(Flight_ID INTEGER PRIMARY KEY, Flight_Name TEXT NOT NULL, Destination TEXT);
CREATE TABLE Booking(Booking_ID INTEGER PRIMARY KEY,
    Passenger_ID INTEGER REFERENCES Passenger(Passenger_ID),
    Flight_ID INTEGER REFERENCES Flight(Flight_ID), Travel_Date TEXT);
INSERT INTO Passenger VALUES
    (1,'Aarthi','Chennai'),(2,'Bharath','Madurai'),(3,'Chandru','Salem'),(4,'Deepa','Trichy');
INSERT INTO Flight VALUES
    (1,'AI-101','Delhi'),(2,'6E-202','Mumbai'),(3,'SG-303','Delhi'),(4,'UK-404','Kolkata');
INSERT INTO Booking VALUES
    (1,1,1,'2026-05-10'),(2,2,2,'2026-05-11'),(3,3,3,'2026-05-12'),(4,4,1,'2026-05-14');
""")

query = """
SELECT p.Passenger_Name, f.Flight_Name, f.Destination, b.Travel_Date
FROM Booking b
JOIN Passenger p ON b.Passenger_ID = p.Passenger_ID
JOIN Flight    f ON b.Flight_ID   = f.Flight_ID
WHERE f.Destination = ?
ORDER BY b.Travel_Date"""

cur.execute(query, ("Delhi",))
show(cur)
con.close()
```

Output:

```
Passenger_Name | Flight_Name | Destination | Travel_Date
------------------------------------------------------------
Aarthi | AI-101 | Delhi | 2026-05-10
Chandru | SG-303 | Delhi | 2026-05-12
Deepa | AI-101 | Delhi | 2026-05-14
```

Viva voce - likely questions:

Q: Why is Destination stored in Flight and not Booking?
A: It is a property of the flight; storing it in Booking would repeat data and break normalisation.

Q: How could the query be made case-insensitive?
A: WHERE LOWER(f.Destination) = 'delhi' or COLLATE NOCASE.

Q: What does the foreign key in Booking guarantee?
A: Every booking refers to an existing passenger and flight (referential integrity).